

Week 2: Product and Category Management System

Scenario

The Organic Grocery Store needs a proper database structure to store and manage its **products and product categories**. Each grocery item is assigned to a particular category such as Fruits, Vegetables, Grains, or Beverages. The system should store important product information including product name, price, and available quantity.

1. Creating Category and Product Tables

To organize the grocery products effectively, two tables are required:

Category Table

The **Category** table stores the different categories available in the grocery store.

```
SQL> CREATE TABLE Category (category_id INT PRIMARY KEY, category_name VARCHAR2(50) NOT NULL UNIQUE, description VARCHAR2(200));  
Table created.
```

Product Table

The **Product** table stores details about the individual grocery items.

```
SQL> CREATE TABLE Product (product_id INT PRIMARY KEY, product_name VARCHAR2(100) NOT NULL, category_id INT, price NUMBER(10,2) CHECK (price > 0), stock_quantity INT CHECK (stock_quantity >= 0), FOREIGN KEY (category_id) REFERENCES Category(category_id));  
Table created.
```

2. Establishing Table Relationships and Constraints

Relationship Between Category and Product

A single category can contain several products.

For example:

- The **Fruits** category can contain Apples, Bananas, and Oranges.
- The **Vegetables** category can contain Carrots, Tomatoes, and Potatoes.

Therefore, the relationship is:

One Category → Many Products (1 : N)

The category_id column is used to establish this connection.

Understanding Keys

Primary Key

A **Primary Key** is an attribute that uniquely identifies every record in a table. It cannot contain duplicate values.

In our tables:

- category_id is the Primary Key of the **Category** table.
- product_id is the Primary Key of the **Product** table.

Foreign Key

A **Foreign Key** is used to establish a connection between two tables.

In the Product table, category_id works as a **Foreign Key** because it refers to category_id in the Category table.

Additional Constraints

NOT NULL

The NOT NULL constraint prevents a column from being left empty.

CHECK

The CHECK constraint ensures that entered values follow a specific condition.

For example, the price of a product should not be zero or negative.

UNIQUE

The UNIQUE constraint prevents duplicate values in a column.

For example, two categories should not have the same category name.

3. Insert Values

```
SQL> INSERT ALL INTO Product VALUES (101, 'Organic Apple', 1, 150, 30) INTO
Product VALUES (102, 'Organic Banana', 1, 80, 50) INTO Product VALUES (103,
'Organic Carrot', 2, 60, 40) INTO Product VALUES (104, 'Brown Rice', 3, 120,
25) INTO Product VALUES (105, 'Green Tea', 4, 200, 20) SELECT * FROM dual;
5 rows created.
```

The Product table contains the grocery items available in the store. Each product is assigned to a particular category using the **Category ID**.

- **Fruits:** Apple and Banana
- **Vegetables:** Carrot
- **Grains:** Rice
- **Beverages:** Green Tea

Each product record includes the **product ID, product name, category ID, price, and stock quantity**. This helps the store easily identify products and know which category they belong to.

Product Records:

```
SQL> INSERT ALL INTO Product VALUES (101, 'Organic Apple', 1, 150, 30) INTO Product VALUES (102, 'Organic Banana', 1, 80, 50) INTO Product VALUES (103, 'Organic Carrot', 2, 60, 40) INTO Product VALUES (104, 'Brown Rice', 3, 120, 25) INTO Product VALUES (105, 'Green Tea', 4, 200, 20) SELECT * FROM dual;

5 rows created.
```

The **Category table** stores the different types of products available in the organic grocery store.

Each category has a unique **Category ID** and a **Category Name**.

For example:

- 1 → Fruits
- 2 → Vegetables
- 3 → Grains
- 4 → Beverages

The Category table helps us **organize products into different groups**. The Category_ID is also used in the Product table to show which category each product belongs to.

5. CRUD Operations on Product Table

CRUD represents the four basic operations performed on database records:

C – Create

R – Read

U – Update

D – Delete

UPDATE OPERATION:

```
SQL> UPDATE Product SET price = 110 WHERE product_id = 106;

1 row updated.
```

The **UPDATE query** is used to **change existing information** in a table.

For example, if we want to change the price of **Organic Apple** from ₹150 to ₹180:

Delete Operation:

```
SQL> DELETE FROM Product WHERE product_id = 106;  
1 row deleted.
```

The **DELETE** query is used to **remove an existing product record** from the Product table.

For example, to remove the product with ID **105**:

DELETE FROM Product → Selects the **Product table**.

WHERE product_id = 105 → Identifies the product that should be removed.

5. Generate Category-wise Product Reports:

SQL queries can be used to generate useful reports based on product categories.

Display Products Along With Their Categories

```
SQL> SELECT c.category_name, p.product_name, p.price FROM Category c JOIN Product p ON c.category_id = p.category_id;
```

CATEGORY_NAME	PRICE	PRODUCT_NAME	
Fruits	150	Organic Apple	
Fruits	0	Organic Banana	8
Vegetables	0	Organic Carrot	6
Grains	120	Brown Rice	
Beverages	200	Green Tea	

This query helps us **see which product belongs to which category along with its price**.

Count Products in Each Category:

```
SQL> SELECT c.category_name, COUNT(p.product_id) AS product_count FROM Category c LEFT JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_name;
```

CATEGORY_NAME	PRODUCT_COUNT
Vegetables	1
Fruits	2
Grains	1
Beverages	1

This query tells us **how many products are present in each category**.

Count Products in Each Category:

```
SQL> SELECT c.category_name, COUNT(p.product_id) AS product_count FROM Category c LEFT JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_name;
```

CATEGORY_NAME	PRODUCT_COUNT
Vegetables	1
Fruits	2
Grains	1
Beverages	1

It counts all the products and shows the **number of products under each category**, such as Fruits, Vegetables, Grains, and Beverages.

Find the Costliest Product in Each Category:

```
SQL> SELECT c.category_name, MAX(p.price) AS highest_price FROM Category c JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_name;
```

CATEGORY_NAME	HIGHEST_PRICE
Vegetables	60
Fruits	150
Grains	120
Beverages	200

This query finds the **maximum product price available in each category**.

Find Categories Containing More Than 5 Products:

```
SQL> SELECT c.category_name, COUNT(p.product_id) AS total_products FROM Category c JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_name HAVING COUNT(p.product_id) > 5;
```

no rows selected

The query is used to identify categories containing more than five products. Since none of the existing categories have more than five products, no rows are displayed.

Calculate Average Price for Each Category:

```
SQL> SELECT c.category_name, AVG(p.price) AS average_price FROM Category c JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_name;
```

CATEGORY_NAME	AVERAGE_PRICE
Vegetables	60
Fruits	115
Grains	120
Beverages	200

This query tells us the **average price of products available in each category**.